JavaScript Project - Form Validation

Back to JS page

Table of Contents

Form Validation Overview

In this project, you will build a sign-up form with input validation.

The form shows inline error messages under each field and prevents submission until all fields are filled and valid.

What You Will Learn

- How to validate user input against custom rules using JavaScript
- How to prevent forms from reloading the page during submit events
- How to write validation check helper functions
- How to check email formats using Regular Expressions (regex)

Step 1 - Create the HTML

Start by creating the basic structure containing four text fields (Name, Email, Password, Confirm Password), associated error message paragraphs, and a submit button.

```
<!DOCTYPE html>
<html>
<body>

<h2>Sign Up</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<form id="signupForm">

<div class="field">
  <label>Name:</label><br>
  <input id="name" type="text" placeholder="Your name">
  <p id="nameError" class="error"></p>
</div>

<div class="field">
  <label>Email:</label><br>
  <input id="email" type="text" placeholder="name@example.com">
  <p id="emailError" class="error"></p>
</div>

<div class="field">
  <label>Password:</label><br>
  <input id="password" type="password" placeholder="Min 8 characters">
  <p id="passwordError" class="error"></p>
</div>

<div class="field">
  <label>Confirm Password:</label><br>
  <input id="confirm" type="password" placeholder="Repeat password">
  <p id="confirmError" class="error"></p>
</div>

<p><button type="submit">Create Account</button></p>

</form>

<p id="result"></p>

<script>
// Create a task Array
let tasks = [];
</script>

</body>
</html>
```

Example 1

Result View Example

Step 2 - Add the CSS

Add styling to distinguish input boxes, error layout styles, and valid confirmation elements:

```
<style>
input {
  padding: 8px;
  width: 260px;
  margin-bottom: 4px;
}
.error {
  color: red;
  margin: 0;
}
.ok {
  color: green;
  margin: 0;
}
.field {
  margin-bottom: 12px;
}
</style>
```

Example 2

Result View Example

Step 3 - Add JavaScript

Cache inputs and build common error layout handlers:

- **Create variables for each element:**

```
const form = document.getElementById("signupForm");
const nameInput = document.getElementById("name");
const emailInput = document.getElementById("email");
const passInput = document.getElementById("password");
const confirmInput = document.getElementById("confirm");
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const passError = document.getElementById("passwordError");
const confirmError = document.getElementById("confirmError");
const result = document.getElementById("result");
```

- **Define message display and clear helpers:**

```
function showError(el, message) {
  el.innerHTML = message;
}

function clearError(el) {
  el.innerHTML = "";
}
```

- **Intercept submits to prevent page reloading:**

```
form.addEventListener("submit", function (event) {
  event.preventDefault();
  result.innerHTML = "";

  if (validateForm()) {
    result.innerHTML = "Form is valid!";
    result.className = "ok";
  } else {
    result.innerHTML = "Please fix the errors.";
    result.className = "error";
  }
});
```

Example 3

Result View Example

Step 4 - Validate Fields

Implement individual validation checks:

- **Name field:** Must contain at least 2 characters.

```
function validateName() {
  let value = nameInput.value.trim();
  if (value.length < 2) {
    showError(nameError, "Name must be at least 2 characters.");
    return false;
  }
  clearError(nameError);
  return true;
}
```

- **Email field:** Verified using a simple regex pattern.

```
function validateEmail() {
  let value = emailInput.value.trim();
  if (!(/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value))) {
    showError(emailError, "Enter a valid email address.");
    return false;
  }
  clearError(emailError);
  return true;
}
```

- **Password field:** Must contain at least 8 characters.

```
function validatePassword() {
  let value = passInput.value;
  if (value.length < 8) {
    showError(passError, "Password must be at least 8 characters.");
    return false;
  }
  clearError(passError);
  return true;
}
```

- **Confirm Password:** Matches the password input.

```
function validateConfirm() {
  let pass = passInput.value;
  let confirm = confirmInput.value;
  if (confirm === "") {
    showError(confirmError, "Please confirm your password.");
    return false;
  }
  if (confirm !== pass) {
    showError(confirmError, "Passwords do not match.");
    return false;
  }
  clearError(confirmError);
  return true;
}
```

- **Final Form validator:**

```
function validateForm() {
  let okName = validateName();
  let okEmail = validateEmail();
  let okPass = validatePassword();
  let okConfirm = validateConfirm();
  return okName && okEmail && okPass && okConfirm;
}
```

Example 4

Result View Example

Exercises & Solutions

Exercise 1

Show an alert or custom styling if something is wrong.

Exercise 2

Check email format using regex (already covered in Step 4).

Exercise 3

Show a green success message next to fields that validate successfully.

Solutions Code

Update display functions to show success styles when parameters match successfully:

```
function showError(el, message) {
  el.innerHTML = message;
  el.className = "error";
}

function showSuccess(el, message) {
  el.innerHTML = message;
  el.className = "ok";
}

function validateName() {
  let value = nameInput.value.trim();
  if (value.length < 2) {
    showError(nameError, "Name must be at least 2 characters.");
    return false;
  }
  showSuccess(nameError, "Name is valid.");
  return true;
}
```

Example 5

Result View Example

Bonus Challenges (Level Up)

Elevate the interface with advanced real-world features:

- **Real-time inline validation:** Bind validators directly to `input` and `blur` events so users receive instant feedback.
- **Complex password rules checklist:** Test and display active checks for length, numbers, and uppercase letters.
- **Password visibility toggler:** Show or hide password inputs dynamically.

Example 6

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Form Validation Project